

SOFTWARE REQUIREMENTS SPECIFICATION

Drone Detection System

Version 1.0

Prepared for Academic Submission

Document Control

Field	Details
Project Name	Drone Detection System
Domain	AI-based surveillance and airspace security
Primary Users	Security operators, administrators, incident reviewers
Status	Final draft

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions and Acronyms
- 2. System Overview
 - 2.1 System Description
 - 2.2 Operating Environment
 - 2.3 User Classes
- 3. Functional Requirements
 - 3.1 Detection and Tracking
 - 3.2 Alerts and Monitoring
 - 3.3 Data Storage and Reporting
- 4. Technology Stack
- 5. System Architecture and High-Level Design
- 6. Low-Level Design
- 7. ER Diagram
- 8. Non-Functional Requirements
- 9. System Features and Working Project Screenshots
- 10. Suggested Improvements and Future Scope
- 11. Conclusion

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification is to define the functional, non-functional, architectural, and operational requirements of the Drone Detection System. The document provides a clear reference for developers, testers, project evaluators, and stakeholders so that all parties understand what the system must do, how it should behave, and what constraints must be considered during implementation.

This SRS is prepared using the structure of the provided SRS format and takes technical reference from the existing Drone Detection System documents. It focuses on a practical academic project scope while keeping the system realistic for security-sensitive environments.

1.2 Scope

Unauthorized drones can create serious risks in airports, military zones, public events, industrial sites, and critical infrastructure areas. They may be used for illegal surveillance, smuggling, restricted-zone entry, or unsafe flying near people and aircraft. The Drone Detection System is designed to detect, track, classify, and report drone activity in real time.

The system uses camera-based computer vision as the primary detection method and can be extended to include radar, RF sensors, thermal cameras, and acoustic sensors. The current scope includes video stream processing, drone identification, alert generation, dashboard display, incident logging, and report generation. The system does not directly perform drone neutralization, but it provides the software foundation required for future counter-drone response integration.

1.3 Definitions and Acronyms

Term	Meaning
UAV	Unmanned Aerial Vehicle, commonly known as a drone
AI	Artificial Intelligence
ML	Machine Learning
YOLO	You Only Look Once, a real-time object detection model
RF	Radio Frequency
PTZ	Pan-Tilt-Zoom camera
C-UAS	Counter-Unmanned Aircraft System
HLD	High-Level Design
LLD	Low-Level Design

2. System Overview

2.1 System Description

The Drone Detection System is a surveillance and alerting solution that analyzes camera frames or video streams to detect drones. When a drone is identified with sufficient confidence, the system displays the result on a dashboard, records the event, and triggers an alert for the operator. The system can also store evidence such as timestamps, detection confidence, frame snapshots, and tracking details for later review.

The reference design supports a multi-layered detection approach. Radar can provide long-range object discovery, RF sensors can detect communication signals between a drone and its controller, and optical or thermal cameras can visually confirm the target. Sensor fusion improves accuracy by correlating multiple data sources and reducing false positives caused by birds, aircraft, or environmental movement.

2.2 Operating Environment

The software may run on a local workstation, an edge device such as NVIDIA Jetson, or a server connected to camera feeds. The dashboard should be accessible through a modern web browser. The backend is expected to run on a Windows or Linux environment with Python support, AI model files, and the required computer vision libraries.

2.3 User Classes

Security operators use the dashboard to monitor live detections and respond to alerts. Administrators configure cameras, detection thresholds, user access, and alert preferences. Investigators or reviewers use stored logs and generated reports to analyze incidents after they occur.

3. Functional Requirements

3.1 Detection and Tracking

- FR-1: The system shall accept image, video, or live camera input for drone detection.
- FR-2: The system shall process frames using an AI-based object detection model such as YOLOv8.
- FR-3: The system shall classify detected objects as drone or non-drone based on model confidence.
- FR-4: The system shall display bounding boxes and confidence scores for detected drones.
- FR-5: The system shall support real-time or near-real-time detection, targeting at least 30 FPS where hardware allows.
- FR-6: The system shall maintain basic tracking information such as time, frame, object location, and detection confidence.
- FR-7: The system should allow future integration with radar, RF, thermal, and acoustic sensors.

3.2 Alerts and Monitoring

- FR-8: The system shall generate an alert when a drone is detected above the configured confidence threshold.
- FR-9: The system shall display alert status on the dashboard with timestamp and severity.
- FR-10: The system shall support audible or visual alert indications for operators.
- FR-11: The system should support SMS, email, or external notification integration in future versions.
- FR-12: The system shall allow operators to review recent detection events from the dashboard.

3.3 Data Storage and Reporting

- FR-13: The system shall store detection logs containing timestamp, confidence value, source, and alert status.
- FR-14: The system shall preserve selected evidence such as screenshots or frame references when required.
- FR-15: The system shall generate reports for incident review and academic demonstration.
- FR-16: The system should support historical replay of detection paths when tracking data is available.

4. Technology Stack

Layer	Recommended Technology
Frontend	React.js or browser-based HTML dashboard
Backend	Python, FastAPI, or Streamlit for monitoring views
AI / ML	YOLOv8, OpenCV, PyTorch
Data Storage	PostgreSQL or SQLite for logs; TimescaleDB for high-frequency telemetry
Communication	WebSockets, MQTT, or REST APIs
Infrastructure	Docker, edge device deployment, optional cloud hosting
Reporting	PDF report generation and structured log exports

The selected technology stack should prioritize low latency, simple deployment, and maintainability. Python is suitable because it integrates well with AI models, OpenCV, and common data-processing libraries. A lightweight frontend or Streamlit dashboard is suitable for academic demonstration, while React and FastAPI are better for a scalable production-style architecture.

5. System Architecture and High-Level Design

The Drone Detection System follows a layered architecture. Each layer has a clear responsibility so the system can be maintained, tested, and extended.

5.1 Perception Layer

This layer contains input devices and sensors. In the current implementation, the camera or video source is the primary input. In a full deployment, this layer can include radar, RF sensors, PTZ cameras, thermal cameras, and microphones.

5.2 Edge Processing Layer

The edge layer performs immediate frame processing and AI inference. It receives the video stream, prepares frames, runs the detection model, filters low-confidence detections, and sends meaningful events to the backend. Edge processing reduces delay and avoids sending unnecessary raw data to a central server.

5.3 Core Application Layer

The core layer manages business logic, event correlation, alert decisions, and storage. It receives detection results, applies configured thresholds, creates incident records, and exposes APIs for the dashboard.

5.4 Presentation Layer

The presentation layer provides the dashboard used by operators. It shows live video, detection results, alert history, confidence values, system status, and reports. The dashboard should be simple, readable, and optimized for quick response.

5.5 High-Level Data Flow

1. Camera or video input provides frames to the processing module.
2. The AI model detects drone-like objects in each frame.
3. The detection engine returns bounding boxes and confidence scores.
4. The alert module checks whether the detection crosses the configured threshold.
5. The dashboard displays results and alerts the operator.
6. The reporting module stores logs and generates incident reports.

6. Low-Level Design

6.1 Core Modules

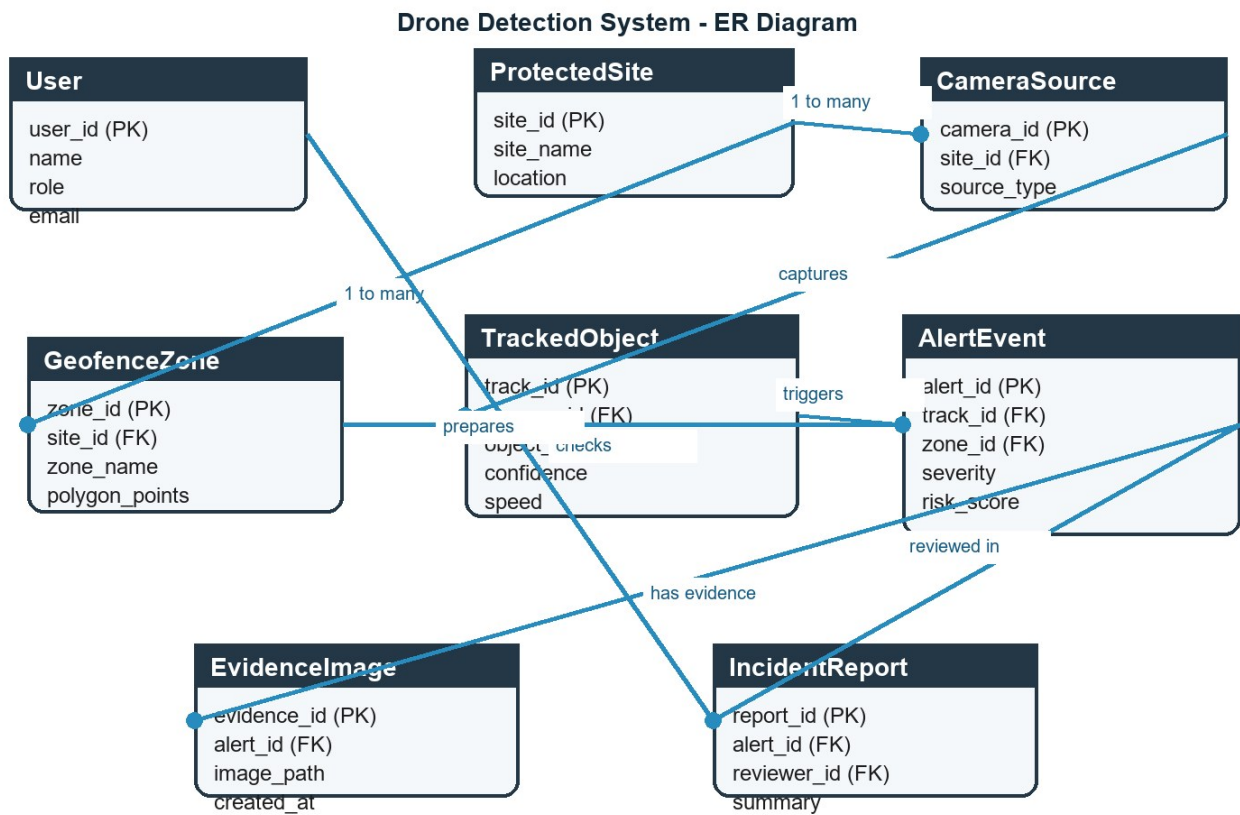
Module	Responsibility
Input Manager	Loads camera feeds, uploaded images, or video files
Detection Engine	Runs YOLO/OpenCV processing and returns detections
Tracker	Maintains object position and movement history
Alert Manager	Generates alerts based on confidence, geofence, or severity
Dashboard	Shows live status, detections, alerts, and reports
Reporting Module	Creates incident summaries and PDF reports
Security Module	Handles access control and configuration protection

6.2 Data Schema Summary

Detection records should include detection ID, timestamp, source name, object class, confidence, bounding box coordinates, alert severity, and operator action. User records should include user ID, role, username, and access permissions. Report records should include report ID, incident time, summary, generated file path, and reviewer notes.

7. ER Diagram

The following ER diagram represents the main data entities used by the Drone Threat Command Dashboard. It covers protected sites, cameras, tracked objects, geofence zones, alert events, evidence images, reports, and users. The structure supports image-based testing, live monitoring, alert review, and incident reporting.



ER Diagram

8. Non-Functional Requirements

8.1 Performance

The system should provide real-time or near-real-time response. For live monitoring, the target processing speed is 30 FPS on suitable hardware. The alert delay should be less than 2 seconds from confirmed detection to dashboard notification.

8.2 Accuracy

The system should achieve high detection accuracy under normal lighting and clear visibility. The target detection rate is above 95 percent in controlled conditions, with a false positive rate below 2 percent after threshold tuning and dataset improvement.

8.3 Availability

The monitoring system should remain available during security operations. For production use, the expected availability target is 99.9 percent or higher, supported by reliable hardware, service monitoring, and backup storage.

8.4 Security

The system shall protect sensitive incident data and camera access. Authentication should be required for administrative functions. Network communication should use encrypted channels such as HTTPS or TLS where applicable. Logs and reports should be protected from unauthorized access.

8.5 Usability

The dashboard should be easy for security operators to understand during stressful situations. Alerts must be visible, timestamps must be clear, and detection information should be presented without unnecessary complexity.

8.6 Maintainability

The system should use modular code so that AI models, input sources, and notification methods can be updated independently. Configuration values such as model path, confidence threshold, and camera source should not require code changes.

9. System Features and Working Project Screenshots

Real-time dashboard: Shows live monitoring status, detections, confidence values, and recent alerts.

AI-based detection: Uses a trained object detection model to identify drones in images or video.

Alert generation: Notifies operators when drone activity is detected above the threshold.

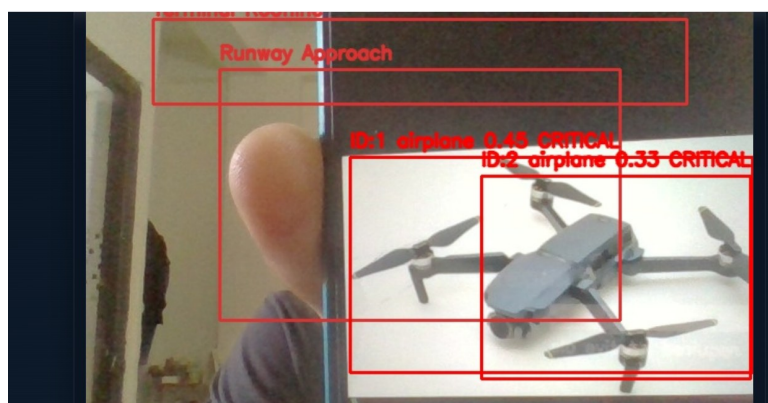
Event logging: Stores detection details for auditing and later investigation.

Report generation: Creates structured incident reports for review and submission.

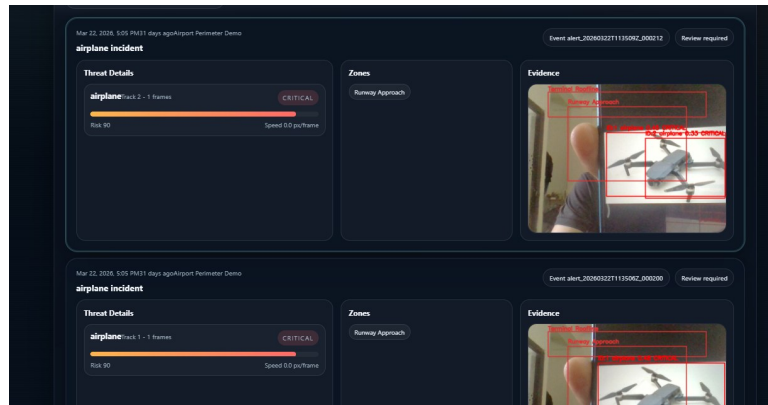
Future sensor fusion: Allows the architecture to expand toward radar, RF, thermal, and acoustic inputs.

Geofencing support: Can be extended to alert only when drones enter restricted virtual zones.

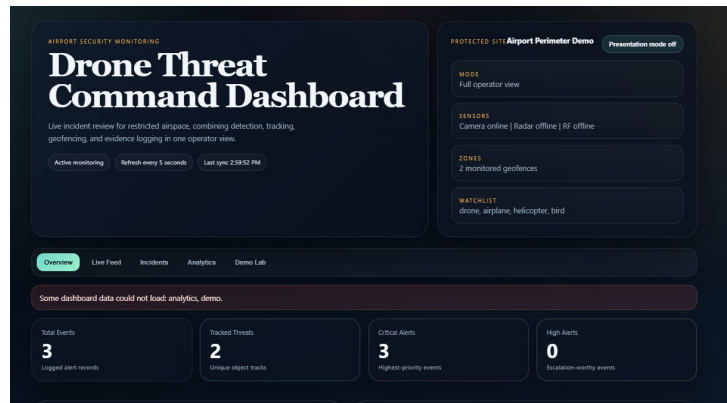
The screenshots below show the working project output. The detector highlights restricted zones and detected objects, assigns threat severity, logs evidence snapshots, and presents incidents through the React dashboard.



Live YOLO detection with geofence overlay



React dashboard incident review with saved evidence



Drone Threat Command Dashboard overview

10. Suggested Improvements and Future Scope

The current system is focused on software-based drone detection and alerting. Future versions can improve reliability, coverage, and operational usefulness through the following enhancements:

1. RF sensor integration can help identify drone-controller communication and may allow operator location estimation.
2. Radar integration can improve long-range detection and work during poor visibility.
3. Thermal camera integration can improve night detection and low-light monitoring.
4. Acoustic sensing can identify drones from propeller sound signatures.
5. Automated geofencing can classify detections by restricted area, risk level, and flight path.
6. Swarm detection logic can detect and track multiple coordinated drones.
7. Mobile dashboard support can allow field personnel to receive alerts quickly.
8. Automated mitigation integration, such as RF jamming or safe interception, may be considered only under legal and authorized conditions.
9. Model retraining with local datasets can improve accuracy for specific environments.
10. Cloud-based analytics can support centralized monitoring across multiple locations.

11. Conclusion

The Drone Detection System provides a practical and scalable solution for identifying unauthorized drones and alerting security personnel in real time. By combining AI-based detection, dashboard monitoring, event logging, and reporting, the system addresses an important modern security challenge.

The proposed architecture supports the current academic implementation while also allowing future expansion into a full multi-sensor counter-drone platform. With proper training data, calibrated thresholds, secure deployment, and continuous improvement, the system can become a reliable tool for airspace monitoring and incident management.